SENG202
Student Guide

Getting started

Welcome to SENG202! This course is structured very differently from those you have done so far. You may have heard about this course or the similar SENG302 that you will be taking next year. There are many similarities between these courses since they both emphasize the creation of software products in large teams, however there is much nuance in the way the outcomes are achieved. Doing well and applying yourself in SENG202 will give you a good starting point for next year when you take SENG302.

Here little emphasis is placed around labs and assignments, instead we focus on the creation of a software product, following the Spiral[1] methodology (a well-known predictive, iterative, and incremental methodology). Of course, this means there is much to learn and apply during the course, so make sure to read through this guide in its entirety and make sure to refer to it later, as many of your questions about the course will be answered. Note that this document may evolve over time.

Throughout the course we will often post on Learn. It is expected that everyone will read all posts in a timely manner. **Not being up to date with these will not be an accepted excuse for missing or incorrect work**.

House Keeping Rules

Professionalism

SENG202 provides a similar experience to working in industry. Because of this you have quite a bit of freedom in how you work, however we expect you to remain professional at all times. This includes class activities and when working with your team even if no teaching staff are present. Also remember that we will be looking at the code you submit to GitLab so keep the code, and any comments, professional.

Continued lack of professionalism may impact assessment.

[1] https://en.wikipedia.org/wiki/Spiral_model

Absences

You are expected to attend all SENG202 sessions every week; this includes a lecture, a lab, and a presentation session. Drop-in sessions are the only exception. If you are absent from a session, you must inform both your team and the teaching team about it beforehand. Recurring absences from other commitments (such as work or sports) will not be accepted. It is your responsibility as a student to ensure you are available for all scheduled class sessions. If you cannot attend a session (i.e. due to sickness), you must inform both the teaching team and your own teammates. Please send absence notices to the shared mailbox lect202@cosc.canterbury.ac.nz and inform your team over your Mattermost channel. Please inform both the teaching team and your team as early as you are able to; this is especially important for longer-term absences. Do note that you do not have to disclose details of the reasons for your absence to your team unless you want to; you are entitled to your privacy here. We expect all students to respect others' boundaries on this and work as a team to help mitigate any issues that arise due to absences in a professional manner.

Working as a Team

The most important part about any large software product is teamwork. A single developer alone does not have the time or resources to create large applications; instead, this work is done by many people, often spanning many different roles. In large companies, teams may even be positioned all around the world, adding many more layers of complexity to the development process. Because of these factors being able to adequately work in a team is of the utmost importance for a software engineer, and for most of you SENG202 will be the first time you undertake a project in a large (~4 to 6 persons) team so it can be a difficult challenge.

At the start of the course, you will be placed into teams of about 4 to 6, depending on the number of students enrolled. These will be the teams you stay in for the rest of the semester, so do try and get to know each other.

Facilitating Better Team Dynamics

We understand that this is likely your first experience working in a large team and do not expect everything to go smoothly. Here is some advice that may help foster better team dynamics:

- Working together

    - Co-located work and pair programming is an important aspect of building relationships within the team. By working together, you become more familiar with your teammates and improve communication. Make sure to include all members of the team and avoid creating 'cliques' within your team where only certain people interact with others, this however does require the team to be accepting of everyone and individuals to make a meaningful effort to interact with others.
- Defining and following processes
    - In the next section, we discuss different team policies. It is imperative that you define these early on and follow them throughout. Setting expectations is important for a highly functional team as you need to be able to trust any other member of your team to complete tasks properly.
    - Failing to meet expectations should not immediately be a cause of concern, however it is important that such instances are discussed, and appropriate actions are taken. The longer you avoid taking action, the harder it will be. Such issues can be caused by many things, such as poorly defined processes, poor communication, or a lack of knowledge. When addressing an issue, it is important to reflect on the cause, not the individual, and look towards a constructive and actionable outcome that benefits the team instead of allowing such issues to keep stacking up or alienating members of your team.
    - In the event of continued failure to meet expectations, it is expected that your team will discuss it with the teaching team to get guidance or help. Communicating issues within your team early is important to resolving them, and we would rather you ask for help than let issues impact your team throughout the whole project.
- Giving and receiving feedback
    - It is important that you are able to give and receive constructive feedback to your team members. As part of professional practice official feedback should be constructive and meaningful, however informal feedback within the team is also important and while we (the teaching staff) may never see or hear it remember to be empathetic. Being able to give members of your team feedback and praise when appropriate is an important aspect of fostering good team dynamics and improving the overall performance of the team.
- Phases of team development (Forming, Storming, Norming, and Performing)
    - Tuckman's model[2] of team development will be discussed in the course, and it is important to understand that, in general, teams go through these different

[2] See https://www.wcupa.edu/coral/tuckmanStagesGroupDelvelopment.aspx for a breakdown of this model

stages. Importantly, do not try to avoid stages such as storming in the hopes that any issues will just disappear, instead it is better to deal with issues and take on the conflict early before it swells up or makes a large impact on your team's performance.

Team Technologies

When working in a team you will need to make use of various technologies for improved communication and organization. You will be provided Mattermost channels which you must use for team communication and basic queries to the teaching team (making sure to tag the most relevant individual). We suggest you use Trello for organization, however it is up to you as a team to make the final decision about organizational technologies you use (other 'commonly' used options are Teams and Google Drive).

Team Policies

When working in a team, it is important that everyone is on the same page, and that the policies for completing work are well-defined and agreed upon. Keeping and referring to a record of these allows everyone to work together more efficiently.

Yellow Card

A yellow card policy is a ruleset put into place by your team as a means to create a single source of truth of the expectations of everyone in your team. These rules define different actions a person can do to incur a yellow card, with different actions taken depending on the number of yellow cards an individual has. It is important for the actions to be specific, for example "Everyone must turn up to meetings on time, and if they cannot make it, they must inform the team on Mattermost at least 30 minutes in advance". Doing so means it is clear when a yellow card should be given. The consequences for a yellow card are up to you as a team to decide, however, they must be non-destructive of team relationships. One such example is "the receiver has to bring snacks to the next meeting". This serves to improve team morale as it not only is a sign of apology from the one bringing the snacks, but it also gives the team an opportunity to share a treat. Consequences should also depend on the 'step', or number of yellow cards given. You may want to start easy and give everyone a 'free' yellow card (since we all make mistakes). From there the actions should increase in severity with the final tier being escalating the issue to the teaching team. You may wish to have a 'decay' time for yellow cards, meaning they will go away after a certain time if no more are received. However, this may depend on the severity you place on the first yellow card, for example one 'free' yellow card over the semester should be lenient enough, but when working in a team over a larger time frame forgiveness for past mistakes is necessary (much like demerit points on your license). A final important note, you must be consistent and use

the yellow card policy every time a rule is broken for it to work. Otherwise, situations may spiral out of control and using them will be much harder as a precedent for ignoring them has already been set. Remember it's not a bad thing to be given a yellow card, as it provides an opportunity for learning and improving.

Definition of Done/Ready

A definition of done (DoD) is a list of conditions that must be met before some process can be considered "done". A similar idea exists for things being considered "ready" (DoR). DoDs apply well to any software development task. For example, you will break your work up into smaller tasks, however each of these tasks needs to be completed to a high standard for everything to go smoothly with your product. Within the scope of this course we suggest having two DoDs, separating them for smaller tasks such as coding and for larger overarching tasks such as preparing for a deliverable submission.

A DoD for a coding task may include:

- The new code follows the team's coding conventions
- All methods and classes have meaningful Javadoc
- The new code has 80% coverage with unit tests
- The code has been reviewed by another team member who was not involved in the task
- . . .

We mentioned the definition of ready (DoR) earlier, this is a similar concept but focusses on something being ready for the next step. As part of a DoD you may require a peer to review the work, here you can include a DoR to specify what is needed for something to be considered ready to review.

Work Expectations

As part of a team, it is important you understand your other team members' commitments and work expectations. In a large team, you will likely measure success in different ways and try to achieve different results. As a start it is good to record other commitments such as work, trainings, and other recurring weekly events you are involved in. You should also include things of larger time-scales such as vacations or trips you are going on, as many of these may not be known at the start of the course it is good to update your team regularly about such events, especially if you will be away and unable to work towards the project or attend meetings. Keeping a good work/life balance is an important part of working on a project over a long time period and will help reduce issues like burnout. Please do take care of yourself and your teammates and discuss with the teaching team if you feel that work is

getting away from you. It is also important to keep this balance and not work too much as it may make others in your team feel like they have to do more to keep up or make others feel like they aren't as involved in the project anymore after large changes/decisions are made by only a few individuals.

Deciding on what to do with your project after SENG202

As this is a group project the Copyright on the produced work is shared, as such you must agree on how you want to license your work. This is important if you want to add the project to your portfolio, include it in job applications, or continue working on it after the course as a passion project. Most teams go with a relaxed approach where any individual can do as they like with the project afterwards. However, make sure to formalize this decision and choose a relevant license. You may also want to include some restrictions such as anonymizing the project data (including the removal of any git artefacts).

Course Structure

Presentation Sessions

Each week your team will take part in a presentation session with half of the teams and the teaching team present.

Presenting

Within this session you will have ~10 minutes (depending on the number of teams) to show the work you have done the previous week, it is up to you to decide what is relevant to discuss and you may make use of slide-sets or demo your application however these are not required (except for the demo during the last presentation). You should focus on what is new since last time and discuss the challenges you went through. This is often helped by showing "real" artefacts. But remember to keep it simple enough that others will understand as they may not have the same context about your project. During the presentation every member must present, and the amount each individual presents should be somewhat equal however it is understandable if there is some variation. Though do note that always contributing only a minimal amount will be noticed, you may decide to discuss as a team how shorter and longer sections can be balanced between individuals over many weeks.

Participation and Feedback

During some of these sessions you will give feedback on the other teams. We will review this feedback and return it to those teams so they can reflect on and improve. It should be

obvious that such feedback must be professional, and any derogatory or offensive remarks will be dealt with by the teaching team. Tips for writing good feedback:

- It can be quite time-constrained to write feedback for the presentation. It is okay to focus on smaller aspects when providing feedback to give something meaningful even if it isn't comprehensive as small and clear aspects are easier for someone to understand and reflect on compared to longer but less focused ones.
- Please provide balanced feedback, listing both strengths and opportunities for improvement.
- Use "I", this allows for the feedback to focus on what you observed instead of assumed interpretations and shows the feedback is your own opinion. For example:
    - "You didn't talk loud enough" vs "I found that I couldn't properly hear you when sitting at the back of the room"
    - "You need to practice more so you are confident and speak louder" vs "I find that when I practice my part before-hand I am more comfortable speaking up, maybe this will also help you"
- Provide actionable feedback; don't simply say they did something wrong. Instead, provide actual things they could do to improve. Often knowing what to do comes from experience and may not translate between individuals exactly but getting feedback including steps others took can be a good start (especially when there is a range of feedback from several individuals). For example:
    - "It was hard to make out what you were saying as you seemed very anxious" vs "I felt that you seemed quite anxious and this impacted your clarity, when I get anxious presenting I try to slow down by taking a deep breath between sentences"
- See the Appendix for some examples of good and bad feedback.

You will receive both team-based and individual feedback from your peers. Please reflect on your presentation skills and try improving bit-by-bit each week.

Lab Sessions

Each week you will have a lab session where a handout will be provided on Learn (each week covering different topics). This session is much like labs you have seen in other courses. Once you complete the exercises in the handout you will complete an associated quiz on the quiz server. These quizzes will only be available for the last hour of the lab session and must be completed. If you are unable to attend a lab session due to sickness, please get in touch with the teaching team and we will schedule a time for you to attempt the quiz. Towards the end of the course, there will no longer be any handouts or associated

quizzes. Instead, these sessions will be for you to co-locate with your team and seek help from the tutors (of course you can ask questions about your project during earlier labs as well).

Deliverables

This course is broken up into 3 separate stages we call "Iterations" with associated "Deliverables" (a product of a development process). Whilst there are 3 different deliverables, each focus on different aspects of your project, building on from each other, and their lengths are not equal. Each deliverable you will submit the relevant aspects of your project following instructions provided in lectures and on Learn. Make sure that you pay attention to these instructions and don't make an incomplete or late submission.

Deliverable 1

The first deliverable is the first version of the 'project document'. This version focuses on your initial requirements, plan, and design. The report will detail your plan for creating the application and the various aspects of requirements engineering that you did to form the plan. You should also include a very early prototype of the application with a few core functions, as well as the project setup check list form completed.

Deliverable 2

In the second deliverable you will continue coding to create your application following the plan you outlined in your 'project document'. During this stage you should focus on developing the Minimum Viable Product (MVP)[3] and deliver a working application that provides some meaningful functionality for the end user. Alongside this we will expect some areas of the 'project document' to be updated and extended.

Deliverable 3

In the final deliverable you will be finishing and polishing your application from deliverable 2 including a final version of your 'project document'. It is important to note that this stage is not as long as the previous one, so it may not be the best idea to plan to implement the same amount of functionality.

[3] A basic version of your application that provides a minimum level of functionality for a user, whilst it may not be 'complete' what is there should provide meaningful functionality such that a user can actually use it.

Logging your Work

Logging your work is vital to communicating and documenting to both your team and the teaching staff what you have accomplished. You are expected to complete ~150 hours of work over the course across a diverse range of tasks, for the teaching team to be able to assess this you need to log all work you do. All work sessions should be an appropriate length <3 hours, if you are finding a task is taking longer than this it may be an indication to stop and look at the problem again or seek help. When logging your work try to make sure there is evidence of the effort you put in, for example if you spend 2 hours coding and do not commit anything that lines up with the time you logged there is no evidence you actually did the work you said. However, the teaching team understands that things don't always work out and you may spend several hours trying to implement something new or fixing a bug without actually making enough progress to commit anything to the codebase, in these cases please do mention the fact you did not achieve anything. That in itself is not bad, instead also think about and include in your log some outcome for example if you struggled to implement some new thing your log may say:

- "I worked to implement function xyz but couldn't get it to work. I have reached out to over Mattermost and discussed pair programming this with them tomorrow"
- "I tried to implement function xyz, however I couldn't get it working because of library abc. I have mentioned to my team that this library is not the correct choice and will discuss with my team about a possible alternative"

Kimai

You must use the following, online time-tracking software: Kimai hosted by the CSSE department here: https://kimai.csse.canterbury.ac.nz/.

What to log

You should log all the time you spend on the project, meeting the expected hours is a hard task if you only log some of your work. There are many different tags included that show the different tasks we expect to see completed, though not all may be relevant to specific deliverables.

How to log

Make sure to log your work as soon as practical after the session, and not log many sessions in a quick burst. Logging after the fact will make it harder to remember what it was you worked on and the amount of time it took. Logging excessive hours is a sign that something is not going right, for example 'crunch time' before submission caused by poor time

management leading up to it. It is for reasons like this that logging many more hours than expected will not necessarily mean you get a high mark. Please refer to the Kimai handout for more information about how to log different activities and other best practice.

When to log

You should log as you go. Logging retrospectively is not allowed: the teaching team will extract the content of logs on a weekly basis. Once the logs of week xyz have been collected by the teaching team, you can no longer log activities for week xyz (or preceding weeks). Logs are collected every Monday at 12 PM.

GitLab

We will provide each team with a UC-hosted Git-Lab repository on https://eng-git.canterbury.ac.nz/ which you will use as a Version Control System (VCS) for your project.

Commit Aliases

We will be analyzing your commits so you must make sure your credentials (or 'alias' as git calls it) line up exactly with your name and uc email, not doing so means your contributions may not be automatically analyzed for marking. There will be more information about this in labs, but if you think there may be something wrong with your git alias please reach out to the tutors during a lab session.

Commit Messages

Much like logging, when you commit to your repository, we expect you to include a meaningful message. This is important from both an evidence perspective and to keep the rest of the team updated with what was changed in a succinct manner. You may wish to decide on a standard as a team, or follow simple rules[4] such as:

- Why have I made these changes?
- What effect have my changes made?
- Why was the change needed?
- What are the changes in reference to?

[4] The following bullet points and more information about commit messages can be found at https://www.freecodecamp.org/news/writing-good-commit-messages-a-practical-guide/

Assessment

This course differs from others you may have taken where all assessment is quantitative, allowing grades to be calculated explicitly (down to 2 decimal places). The assessment profile in SENG202 includes qualitative items so you, and your team, will receive letter grades for each deliverable. You should think of these as snapshots indicating your current level of performance. This is similar to the way other CSSE project courses and theses are assessed. This approach allows us to reward students more accurately for the technical progress and professional growth they have displayed throughout the semester.

Deliverables

Each of the deliverables you submit will be marked as a team. We will attempt to provide prompt feedback on these submissions. However, do note that marking large reports and code bases is a very time-consuming task. We encourage you to discuss issues with the teaching team as they arise rather than wait for formal feedback to be given. The grade your team receives for a deliverable will influence the individual grade you receive; however other factors will be taken into account as discussed below.

Labs and Bonuses

All labs are compulsory, and each is marked making up 15% of the course grade. There is also a series of exercises on empathy making up 5% of the course, and this is a all or nothing mark. You will hear more about this during an early lecture.

Individual Grading

As mentioned above, your individual grades will be influenced by the grade your team receives for the deliverable. However, this will be affected by your contributions to the team, including:

- Number of hours (properly) logged
- Contributions to the project document and/or the codebase
- The diversity of contributions made
- Presentation ability in the deliverable (formal) presentation sessions
- Professional behaviour

Overall

Your overall grade in the course will be made up of your individual grade for each deliverable in addition to labs and the meeting communication exercise. As mentioned above the grades for the deliverables can evolve during the course to better reflect a student's overall

performance. Finally, do note that there are some aspects we take into account overall, which may affect your grade on a larger scale. For instance, if a student shows no evidence of trying to meet the expected hours.

Getting Help

The SENG202 teaching team is here to help including the lecturers and tutors, some questions may be more relevant for some individuals to answer for example questions about implementation details may best be asked in lab sessions while higher-level questions about the course will be better answered by the lecturer or course coordinator.

The best place to ask questions will normally be in your team's "Teaching Team Communications" Mattermost channel, make sure to tag the most relevant member of the teaching team otherwise they might not see it! Where you have specific queries or concerns that are not appropriate to be shared with your team please email lect202@canterbury.ac.nz, making sure to include your team number in the subject line. All information you share with the teaching team will remain confidential and will not be shared with your team or others without your express permission. When asking for help we expect that you have:

1. tried to solve the problem yourself (or as a team);
2. tell us about what you did. A great way to do this is through 'Rubber Ducking'[5] by doing this you can include a simple overview of what the issue is and what you tried. Following such a technique will often either make it so you can find the answer yourself, or if help is needed the teaching staff can give more accurate advice more quickly as more is known about the problem. Here are a couple of examples of what you might send:
    a. Course info – "I am trying to work out how exactly I should log my time when I was making some changes someone requested. I had a look at the Kimai handout but it only mentioned the 'fix' tag which doesn't feel right to me since it wasn't actually a bug I was fixing"
    b. Finding a library – "I am looking for a library to help save and load data from JSON files. I tried NotARealJsonLibrary but that didn't work on the version of Java we are using"

[5] https://www.freecodecamp.org/news/rubber-duck-debugging/

Getting feedback

An important aspect of doing well in this course is continuously trying to improve. Even if not all the things you try are successful, learning what doesn't work is a step in the right direction and allows for reflecting on what didn't work to try something new. Due to the nature of the course, you can go quite a while without official feedback, so we suggest teams and individuals alike seek feedback early and often. The second half of the presentation session is a good opportunity to discuss your current progress with the teaching team and get feedback. However you can also schedule longer sessions with your team if you want to get deeper feedback. Getting personal feedback is also important if you feel that there are areas where you need to improve or want to go over feedback provided from a deliverable in more detail.

Appendix

Notes about the use of GenAI

GenAI (e.g. ChatGPT), whilst still somewhat new, can be an invaluable tool for software engineers as it can be immensely helpful in helping understand and write code. However, industry expects that you will have honed your development skills and are able to produce code without the support of GenAI. They would also expect that you are capable of critically assessing the quality of source code produced by an AI. To meet these expectations, you must develop Software Engineering skills without relying on generative AI (too much). In fact, an over-reliance on generative AI without properly understanding/reviewing the code it produces can lead to massive amounts of issues (at least at the time of writing). The daily volume of code produced by GenAI can become overwhelming very rapidly; even for an expert programmer. GenAI can also produce a sense of false confidence that a code is correct; even for an expert programmer. This may lead to situations where a significant amount of code is produced, considered correct at a given point in time, but require major re-engineering at a later date. If this is true for a team of experienced programmers with a good domain knowledge, what can happen to a team of junior programmers with no domain knowledge?

Ask ChatGPT: "Answer yes and why: Would you recommend a team of students learning how to program as a team for the first time, and who start from scratch with no domain knowledge to use generative AI as a significant producer of the requirements, design, and source code of their software application?" Please let the teaching team know if the answer is no, or if the explanation is convincing. The teaching team has tried…

On the other hand, there is no equal for learning and solving problems on your own than making use of any available resources whether it be Google or ChatGPT … **IF** you understand and validate what it produces. Given the volume produced by GenAI, and how persuasive GenAI is, this is a **big IF**. Over relying on such tools can be a very slippery slope… Especially for beginners.

For these reasons, the use of GenAI is restricted to brainstorming, learning (e.g. ask GenAI to ask you to you solve a problem to learn how to do it), and documenting yourself … But none of the material produced by an AI can be part of any of your assessments. You must write the code, tests, documents, presentations, etc.

As part of the deliverable presentation sessions, the teaching team may ask to explain any aspect of the code you delivered. Failing to explain a piece of code properly would significantly impact your deliverable mark.

Reference Material

Creating Diagrams

In your project document, you will need to include diagrams. Whilst many different tools exist online for creating the different diagrams, we suggest making use of PlantUML (you will be using it in SENG301 next year so why not get the head start). A handout that discussed the basics of PlantUML was made available to you in SENG201, which we have also included in the SENG202 resources.

Labs

Other important information will be distributed in the form of labs throughout the course, however we will try to release the handouts for these early so that those who are interested can get more information about topics such as Git, Acceptance Testing, Mapping, etc. . . early.

Online Resources

There are many online resources that can help with the things to learn and apply in the course, here are a few of those that we think are a good start. However throughout the course you will likely learn the usefulness of being able to research and solve your own issues using Google (or the newer ChatGPT)

- Java and JavaFX – Baeldung (Java tutorials and examples) https://www.baeldung.com/java-tutorial – JavaFX https://openjfx.io/
- Git – A handy Git 'cheat sheet' https://www.atlassian.com/git/tutorials/atlassian-git-cheatsheet – GitLab tutorials https://docs.gitlab.com/ee/tutorials/

- Build Tools (Gradle) – Gradle Documentation https://docs.gradle.org/current/userguide/userguide.html
- APIs – Open Route Service https://openrouteservice.org/
    - Google Cloud Console https://console.cloud.google.com/
    - UC Maps API https://tile.csse.canterbury.ac.nz/

Example Feedback

Here are some curated examples of feedback written by ChatGPT focusing on different issues that can impact feedback and how they can be re-written in a professional and constructive way. It should be clear that feedback like that shown in the poorly written section is not acceptable in this course.

Poorly written:

- Blaming the individual - "Your presentation was a disaster. It was evident that you didn't put any effort into preparing for it. You seemed completely lost and unprofessional. You need to take this more seriously and stop wasting everyone's time."
- Lack of constructive criticism - "Your presentation was terrible. It was boring and lacked any substance. You need to step up your game and learn how to engage the audience. Your overall performance was a disappointment."
- Personal attacks - "You are a terrible presenter. Your speaking skills are abysmal, and you have zero charisma. It was painful to listen to you stumble through your words. Maybe public speaking is just not your thing, and you should consider a different career path."
- Dismissive and unhelpful feedback - "Your presentation was a complete waste of time. I didn't learn anything, and I regret attending. You should seriously reconsider your approach because it was utterly ineffective."

Fixed:

- Blaming the individual - "I noticed a few areas in your presentation where more preparation could have been helpful. By dedicating some extra time to rehearsing and refining your content, you can enhance your confidence and deliver a more polished presentation."
- Lack of constructive criticism - "While watching your presentation, I felt that there were opportunities to make the content more engaging and memorable. Consider incorporating more visual aids or interactive elements to captivate the audience's attention and reinforce your main points."

- Personal attacks - “Your presentation could benefit from further development in terms of your delivery and stage presence. By honing your speaking skills and exploring techniques to project more confidence, you can create a more impactful and professional impression on the audience.”
- Dismissive and unhelpful feedback - “I think there is potential for improvement in your presentation. It would be beneficial to analyze the areas where it fell short and identify strategies to make it more informative and engaging for future audiences.”